Day(43)

*Special Functions in python:-

→ In python programming, we have 3 types of special functions.
They are:- ① filter()
② map()
③ reduce().

① Filter()

→ Filter() is used for "filtering out some elements from list of elements by applying to function".

syntax:

```
varname = filter(functionname, iterable_object)
```

=> Here 'varname' is an object of type <class,'filter'> and we can convert into any iterable object by using typecasting functions.

=> "FunctionName" represents either normal function or anonymous function.

=> "iterable_object" represents sequence, list, set and dict types.

=> The execution process of filter() is that "Each value of iterable object sends to function Name.

- if the function returns True then the element will be filtered. if the function returns false then that element will be neglected/not filtered. This process will be continued until all elements of iterable object completed.

program for accepting list values and filter only Numerical Integer value.

Ex①

```
def getintvalues(value):  #normal function Definition.
    if (value[0]=="-") and (value[1:].isdigit()):
        return = True.
    elif (value.isdigit()):
        return True
    else:
        return False
#main program.
print ("Enter list of values separated by space:")
lst = [val for val in input().split()] # ['10','nossum','34',
                                       'True','2+3j',34.56']
print ("Content of lst=", lst)
vals = filter (getintvalues, lst))
# convert filter object into list
intvals = list (vals)
print ("List of integers=", intvals)
```

Ex②

```
getintvalues = lambda value: True if (value[0]=="-") and
                (value[1:].isdigit()) else True
                if value.isdigit() else False
#main program.
vals = tuple (filter ( सेट का नाम)
print ("List of integers=", vals)
```

Ex(3)
```
print ("Entr list of values separated by space:")
lst ← asitis.
print ← asitis.
vals = tuple (filter(lambda value: True if (value[0]=="-")
    and (value[1:].isdigit()) else True if value.isdigit()
    else false, lst))
print ("list of integers =", vals)
```

Ex(4)
```
#program for Reading list of values and obtains +ve values.
def pos(n):
    if (n>0):
        return True
    else:
        return false
#main program
print ("Entr list of values separated by comma:")
lst = [float(val) for val in input().split(",")]
print ("given list of values =", lst)
posvals = list(filter(pos, lst))
print ("list of +ve values=", posvals)
```

(a)

Ex(5)
(b)
```
pos = lambda value: value>0
#main program
same as it is.
```

Ex(6)
```
posvals = list(filter(lambda n: n>0, lst))
```
बाकी सब same.

n<0

Ex(7) बाकी सब same Hai.

(2) map()

→ map() is used for obtaining new iterable object from existing iterable object by applying old iterable elements to the function.

→ In otherwords, map() is used for obtaining new list of elements from existing list of elements by applying oldlist element to the function.

Syntax: varname = map(functionname, iterable object)

=> Here 'varname' is an object of type <class,'map'> and we can convert into any iterable object by using type casting functions.

=> "FunctionName" represents either Normal function or Anonymous functions.

=> "iterable_object" represents sequence, list, set and dict type.

=> the execution process of map() is that "map() sends every elements of iterable_object to the specified function, process it and returns the modified value (result) and new list of elements will be obtained". This process will be continued until all elements of iterable_object completed.

#program for accepting list of values and obtain squares.

ex①.

```
def squares(value):
    return(value**2)
#main program
print("Enter list of values separated by space:")
lst = [float(val) for val in input().split() if val.isdigit()]
mapobj = map(squares,lst)
print("type of sqlist=", type(mapobj))
#convert map objects into lists.
sqlist = list(mapobj)
print("Given list=", lst)
print("squares list=", sqlist)
```

ex(2)

```
square=lambda value: value**2.
#main program.
print lst  same ex(1).
sqlist=tuple(map(squares,lst))
sqrtlist=list(map(lambda value:round(value**0.5,2),lst))
print("---")
print("Given lists=",lst)
print("square lists=",sqlist)
print("square root list=",sqrtlist)
```

ex(3)

```
# program for accepting list of salaries and give 50% Hike to every employee.
print("Enter list of employee salaries:")
sallist=[float(sal) for sal in input().split()]
newsallist=list(map(lambda sal: sal+sal*50/100, sallist))
print("*"*50)
print("old salary list \t\t new salary list")
print("*"*50)
for osl,nsl in zip(sallist,newsallist):
        print("\t{}\t\t{}".format(osl,nsl))
print("*"*50)
```

ex(4)

```
# program for computing sum of two list objects where they must contain equal numbers
print("Enter list of values for first list separated by space:")
lst1=[float(val) for val in input().split()]
repeat करेंगे इस list के लिए.
#Expected output lst3=[110,220,330,440,550]
lst3=list(map(lambda x,y:x+y,lst1,lst2))
print("*"*50)
print("first list \t\t second list \t\t sum list")
print("*"*50)
for fl,sl,rs in zip(lst1,lst2,lst3):
        print("\t{}\t\t\t{}\t\t\t{}".format(fl,sl,rs))
print("*"*50)
```

Q5)

```
#Program for computing total sal of two list objects.
#where they must contain salary & Bonus.
print("Enter list of salaries for employees separated by space:")
sallst = [float(val) for val in input().split()]
print("Enter list of Bonus for employee separated by space:")
bonuslist = [float(val) for val in input().split()].

#Case I first list elements are more than second list elements
 and substitute 0 for remaining elements in second list
if (len(sallst) > len(bonuslist)):
        for i in range (len(sallst) - len(bonuslist)):
                bonuslist.append(0)
elif (len(bmuslist) > len(sallst)):
        for i in range (len(bonuslist) - len(sallst)):
                sallst.append(0)

totlst = list(map(lambda sal, bmus: sal+bmus, sallst,
                                        bmuslist))
print("*"*50)
print("Emp salary \t\t Emp Bonus \t\t total salary")
print("*"*50)
for sal, bmus, tot in zip(sallst, bmuslist, totlst):
        print("\t{}\t\t\t{}\t\t\t{}".format(sal, bmus, tot))
print("*"*50).
```

Day (49)

(3) reduce() :-

→ reduce() is used for obtaining a single element / result from given iterable object by applying to a function.

syntax:

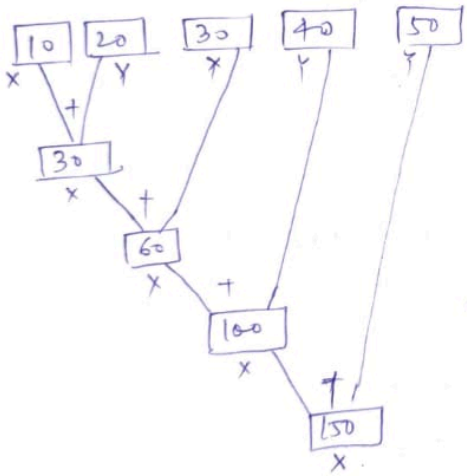
Varname = reduce(function-name, iterable_object)

=) Here 'varname' is an object of int, float, bool, complex, str only

=) the 'reduce()' belongs to a predefined module called "~~function~~ functools"

→ Consider the Following :—

lst1 = [10, 20, 30, 40, 50] --- find sum using reduce()



Since reduce() don't have next element in iterable object Hence reduce function returns value of First variable --- x → 150.

Logic ①:

```
import functools
res = functools.reduce(
      lambda x,y : x+y, lst1)
print(res)
```

Logic ②:

```
def sumop(x,y):
    return(x+y)
res = functools.reduce
    (sumop, lst1)
print(res)
```

Internal flow of reduce:-

step①:- initially, reduce() selects first two values of iterable object and place them in first var and second var.

step②:- The function-name (lambda or normal function) utilizes the values of first var and second var and applied to specified logic and obtains the result.

step③:- reduce() places the result of function-name in first variable and reduce() selects the succeeding element of iterable object and places in second variable.

step④:- Repeat step② and step③ untill all elements completed in iterable object & returns the result of first variable.

program to demonstrate reduce function

ex①.

```
import functools
def sumop(x, y):
    return (x+y)
#main program
print ("Enter list of value separated by space:")
lst = [int(val) for val in input().split()]
print ("  -  -  -  ")
print ("Given list = {}".format(lst))
res = functools.reduce (sumop, lst)
print ("sum({}) = {}".format (lst, res))
print ("  -  -  -  -  -")
```

values

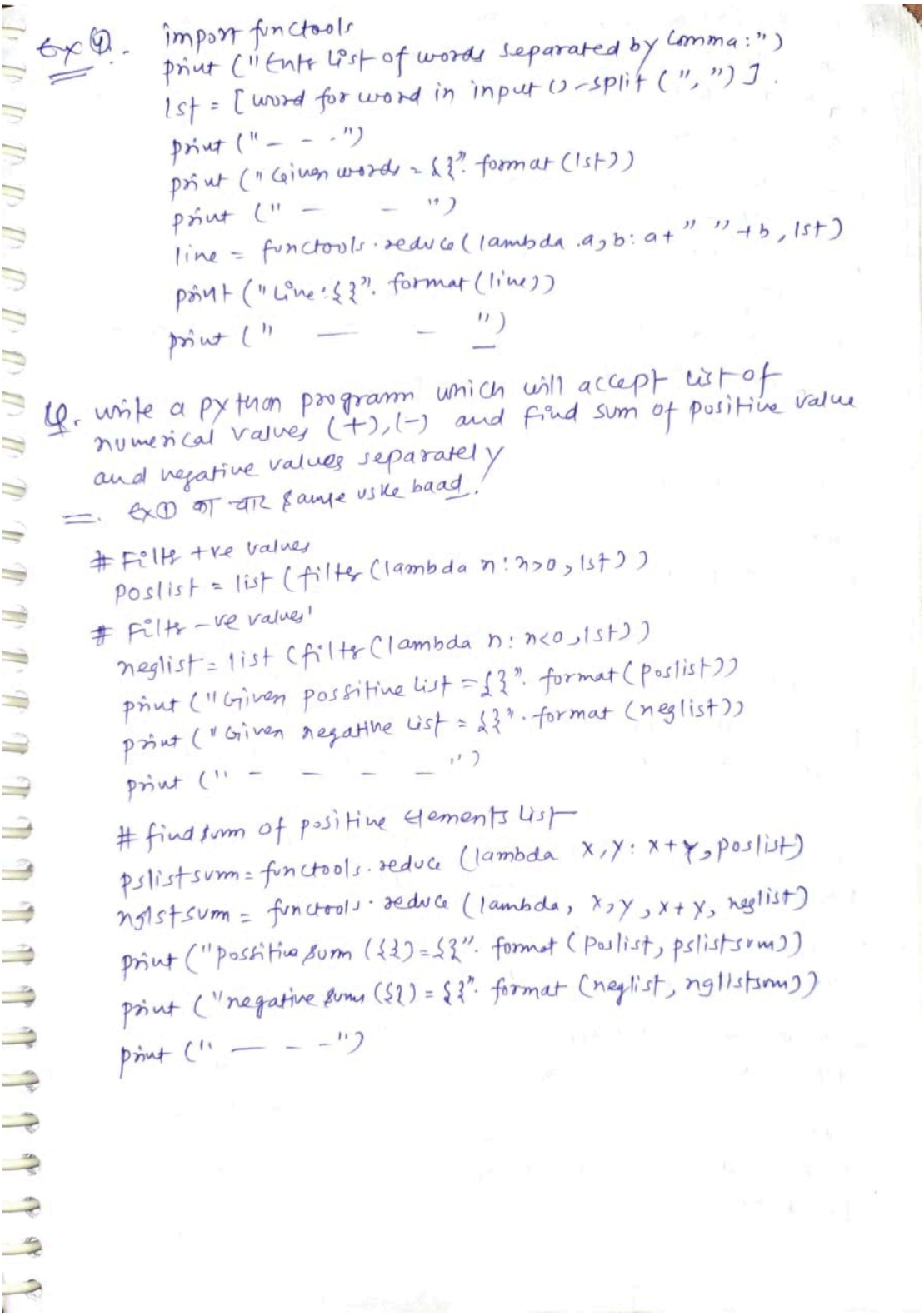
ex②. Upar me import karne ke baad aisa hi.

```
res = functools.reduce (lambda x, y : x : y, lst)
```

Uske baad aisa hi.

ex③. Isme bhi same.

```
big Y = functools.reduce (lambda x, y: x if x > y else y, lst)
small Y =                                  x < y.
print ("big({}) = {}".format (lst, big Y))
print ("small({}) = {}
```



```
Ex(4).- import functools
print("Enter list of words separated by comma:")
lst = [word for word in input().split(",")].
print("---")
print("Given words = {}".format(lst))
print("-------")
line = functools.reduce(lambda a,b: a+" "+b, lst)
print("Line:{}".format(line))
print("-------")
```

Q. write a python program which will accept list of numerical values (+),(-) and find sum of positive value and negative values separately

=. Ex(1) का वाला same use करके बाद.

```
# Filter +ve values
poslist = list(filter(lambda n: n>0, lst))
# Filter -ve values
neglist = list(filter(lambda n: n<0, lst))
print("Given possitive list = {}".format(poslist))
print("Given negative list = {}".format(neglist))
print("-----------")
# find sum of positive elements list
pslistsum = functools.reduce(lambda x,y: x+y, poslist)
nglstsum = functools.reduce(lambda x,y: x+y, neglist)
print("possitive sum ({}) = {}".format(poslist, pslistsum))
print("negative sum ({}) = {}".format(neglist, nglistsum))
print("-----")
```

special function in python:

- 1.filter()
- 2.map()
- 3.reduce()

1.filter()

- syntax:- varname=filter (functionname,iterable_object)

In [6]:
```
#Program for Accepting List Values and Filter Only Numerical Integer Values.
#ex1.py
def getintvalues(value): # Normal Function Definition
    if(value[0]=="-") and (value[1:].isdigit()):
            return True
    elif(value.isdigit()):
        return True
    else:
        return False
#Main Program
print("Enter List of Values separated by space:")
lst=[val for val in input().split()]#['10', 'Rossum', '34', 'True', '2+3j', '34.
print("Content of lst=",lst)
vals=filter(getintvalues,lst)
print("Type of vals=",type(vals))
#Convert filter object into list
intvals=list(vals)
print("List of Integers=",intvals)
```

```
Enter List of Values separated by space:
Content of lst= ['10', 'rossum', '34', 'True', '2+3j', '34.56']
Type of vals= <class 'filter'>
List of Integers= ['10', '34']
```

In [7]:
```
#Program for Accepting List Values and Filter Only Numerical Integer Values.
#Ex2.py
getintvalues=lambda value: True if(value[0]=="-") and (value[1:].isdigit()) else
#Main Program
print("Enter List of Values separated by space:")
lst=[val for val in input().split()]
print("Content of lst=",lst)
vals=tuple(filter(getintvalues,lst))
print("List of Integers=",vals)
```

```
Enter List of Values separated by space:
Content of lst= ['10', 'rossum', '34', 'True', '2+3j', '34.56']
List of Integers= ('10', '34')
```

In [8]:
```
# Ex3:
print("Enter List of Values separated by space:")
lst=[val for val in input().split()]
print("Content of lst=",lst)
vals=tuple(filter(lambda value: True if(value[0]=="-") and (value[1:].isdigit())
print("List of Integers=",vals)
```

```
Enter List of Values separated by space:
Content of lst= ['10', 'rossum', '34', 'True', '2+3j', '34.56']
List of Integers= ('10', '34')
```

In [9]:
```
#Program for Reading List of Values and Obtains +ve Values
#ex4:
def pos(n):
    if(n>0):
        return True
    else:
        return False
#Main Program
print("Enter List of Values Separated By Comma:")
lst=[float(val) for val in input().split(",")] # lst=[10,-23,34.56,-5.6,23,-45]
print("Given List of Values=",lst)
```

```
posvals=list(filter(pos,lst))
print("List of +VE Values=",posvals)
```

```
Enter List of Values Separated By Comma:
Given List of Values= [10.0, -23.0, 34.56, -5.6, 23.0, -45.0]
List of +VE Values= [10.0, 34.56, 23.0]
```

In [11]:
```
#Program for Reading List of Values and Obtains +ve Values
#FilterEx5
pos=lambda value: value>0
#Main Program
print("Enter List of Values Separated By Comma:")
lst=[float(val) for val in input().split(",")] # lst=[10,-23,34.56,-5.6,23,-45]
print("Given List of Values=",lst)
posvals=list(filter(pos,lst))
print("List of +VE Values=",posvals)
```

```
Enter List of Values Separated By Comma:
Given List of Values= [10.0, 23.0, 34.56, -5.6, 23.0, -45.0]
List of +VE Values= [10.0, 23.0, 34.56, 23.0]
```

In [12]:
```
#Program for Reading List of Values and Obtains +ve Values
#FilterEx6
print("Enter List of Values Separated By Comma:")
lst=[float(val) for val in input().split(",")] # lst=[10,-23,34.56,-5.6,23,-45]
print("Given List of Values=",lst)
posvals=list(filter(lambda n: n>0 , lst))
print("List of +VE Values=",posvals)
```

```
Enter List of Values Separated By Comma:
Given List of Values= [10.0, -23.0, 34.56, -5.6, 23.0, -45.0]
List of +VE Values= [10.0, 34.56, 23.0]
```

In [13]:
```
#Program for Reading List of Values and Obtains +ve Values
#FilterEx7
print("Enter List of Values Separated By Comma:")
lst=[float(val) for val in input().split(",")] # lst=[10,-23,34.56,-5.6,23,-45]
print("Given List of Values=",lst)
posvals=list(filter(lambda n: n<0 , lst))
print("List of +VE Values=",posvals)
```

```
Enter List of Values Separated By Comma:
Given List of Values= [10.0, -23.0, 34.56, -5.6, 23.0, -45.0]
List of +VE Values= [-23.0, -5.6, -45.0]
```

2.map()

- syntax:- varname=map(functionname,iterable_object)

In [14]:
```
#Program for accepting list of Value and Obtains Squares
#MapEx1.py
def squares(value):
    return value**2

#Main Program
print("Enter List of Values separated by space:")
lst=[ float(val) for val in input().split() if val.isdigit()]
mapobj=map(squares,lst)
print("Type of sqlist=",type(mapobj))
#Convert Map Object into List
```

```
sqlist=list(mapobj)
print("Given List=",lst)
print("Square List=",sqlist)
```

```
Enter List of Values separated by space:
Type of sqlist= <class 'map'>
Given List= [56.0, 67.0, 87.0, 68.0, 98.0, 76.0]
Square List= [3136.0, 4489.0, 7569.0, 4624.0, 9604.0, 5776.0]
```

In [15]:
```
#Program for accepting list of Value and Obtains Squares
#MapEx2.py
squares=lambda value: value**2
#Main Program
print("Enter List of Values separated by space:")
lst=[ float(val) for val in input().split() if val.isdigit()]
sqlist=tuple(map(squares,lst))
sqrtlist=list(map(lambda value:round(value**0.5,2),lst))
print("---------------------------------")
print("Given List=",lst)
print("Square List=",sqlist)
print("Square Root List=",sqrtlist)
print("---------------------------------")
```

```
Enter List of Values separated by space:
---------------------------------
Given List= [56.0, 65.0, 87.0, 76.0, 22.0, 32.0, 21.0, 23.0, 45.0]
Square List= (3136.0, 4225.0, 7569.0, 5776.0, 484.0, 1024.0, 441.0, 529.0, 2025.
0)
Square Root List= [7.48, 8.06, 9.33, 8.72, 4.69, 5.66, 4.58, 4.8, 6.71]
---------------------------------
```

In [16]:
```
#Program for accepting list of salaries and give 50% hike to Every Employee
#ex3:
print("Enter List of Employee Salaries:")
sallist=[float(sal) for sal in input().split()]
newsallist=list(map(lambda sal: sal+sal*50/100, sallist))
print("*"*50)
print("Old Salary List\t\tNew Salary List")
print("*"*50)
for osl,nsl in zip(sallist,newsallist):
    print("\t{}\t\t{}".format(osl,nsl))
print("*"*50)
```

```
Enter List of Employee Salaries:
**************************************************
Old Salary List         New Salary List
**************************************************
        8000.0          12000.0
        9000.0          13500.0
        7000.0          10500.0
        6000.0          9000.0
        5000.0          7500.0
        3000.0          4500.0
**************************************************
```

In [18]:
```
#Program for Computing Sum of Two List Objects where they Must contain Equal Ele
#MapEx4.py
print("Enter List of Values for First List Separated by Space:")
lst1=[float(val) for val in input().split()]
print("Enter List of Values for Second List Separated by Space:")
lst2=[float(val) for val in input().split()]
```

```
#Expected Output lst3=[110,220,330,440,550]
lst3=list(map(lambda x,y:x+y, lst1,lst2 ))
print("*"*50)
print("First List\t\tSecond List\t\tSum List")
print("*"*50)
for fl,sl,rs in zip(lst1,lst2,lst3):
    print("\t{}\t\t\t{}\t\t\t\t{}".format(fl,sl,rs))
print("*"*50)
```

```
Enter List of Values for First List Separated by Space:
Enter List of Values for Second List Separated by Space:
**************************************************
First List              Second List             Sum List
**************************************************
        54.0                    54.0                            108.0
        45.0                    81.0                            126.0
        41.0                    82.0                            123.0
        42.0                    86.0                            128.0
        43.0                    83.0                            126.0
        48.0                    87.0                            135.0
        49.0                    89.0                            138.0
**************************************************
```

In [19]:
```
#Program for Computing total sal of Two List Objects
# where they Must contain Salary and Bonus
#MapEx5.py
print("Enter List of Salaries for Employees Separated by Space:")
sallst=[float(val) for val in input().split()]
print("Enter List of Bonus for Employee Separated by Space:")
bonuslist=[float(val) for val in input().split()]
#Case-1  First List Elements are More then Second List Elements and substitute 0
if(len(sallst)>len(bonuslist)):
    for i in range(len(sallst)-len(bonuslist)):
        bonuslist.append(0)
elif (len(bonuslist)>len(sallst)):
    for i in range(len(bonuslist)-len(sallst)):
        sallst.append(0)
totlst=list(map(lambda sal,bonus:sal+bonus,sallst,bonuslist))
print("*"*50)
print("Emp Salary\t\tEmp Bonus\t\tTotal Salary")
print("*"*50)
for sal,bonus,tot in zip(sallst,bonuslist,totlst):
    print("\t{}\t\t\t{}\t\t\t\t{}".format(sal,bonus,tot))
print("*"*50)
```

```
Enter List of Salaries for Employees Separated by Space:
Enter List of Bonus for Employee Separated by Space:
**************************************************
Emp Salary              Emp Bonus               Total Salary
**************************************************
        4000.0                  100.0                           4100.0
        5000.0                  200.0                           5200.0
        6000.0                  300.0                           6300.0
        7000.0                  400.0                           7400.0
        8000.0                  500.0                           8500.0
        9000.0                  600.0                           9600.0
**************************************************
```

3.reduce()

- syntax:- varname=reduce(functionnames,iterable_object)

In [20]:
```
#Program for Finding Sum of List of Numbers by using recduce()
#ReduceEx1.py
import functools
lst=[10,20,30,40,50]
res=functools.reduce(lambda x,y: x+y, lst)
print("sum({})={}".format(lst,res))
```

```
sum([10, 20, 30, 40, 50])=150
```

In [21]:
```
#Program for Finding Sum of List of Numbers by using recduce()
#ReduceEx2.py
import functools
def sumop(x,y):
    return (x+y)

#Main Program
print("Enter List of Values Separated by space:")
lst=[float(val) for val in input().split()]
res=functools.reduce(sumop, lst)
print("sum({})={}".format(lst,res))
```

```
Enter List of Values Separated by space:
sum([45.0, 54.0, 71.0, 17.0, 12.0, 21.0, 23.0, 32.0, 25.0])=300.0
```

In [22]:
```
#Program for Finding max and min of List of Numbers by using recduce()
#ReduceEx3.py
import functools
print("Enter List of Values Separated by space:")
lst=[float(val) for val in input().split()] # lst=[10,20,14,30,5]
maxv=functools.reduce(lambda k,v: k if k>v else v, lst)
minv=functools.reduce(lambda k,v: k if k<v else v, lst)
print("Max({})={}".format(lst,maxv))
print("Min({})={}".format(lst,minv))
```

```
Enter List of Values Separated by space:
Max([10.0, 20.0, 14.0, 30.0, 5.0])=30.0
Min([10.0, 20.0, 14.0, 30.0, 5.0])=5.0
```

In [23]:
```
#Program for obtaining a line of text from List of words
#ReduceEx4.py
import functools
print("Enter List of Words Separated by Comma:")
lst=[val for val in input().split(",")] # lst=["Python","Is","An","oop","Lang"]
line=functools.reduce(lambda x,y: x+" "+y, lst)
print("Line of text=",line)
```

```
Enter List of Words Separated by Comma:
Line of text= python is an oop lang
```

In [ ]: